

TaskFlow

Project Technical Report

ASP.NET Core MVC · SQLite · In-Memory Architecture

Generated: May 06, 2026

Version	Framework	Database	Auth
1.0.0	.NET 8	SQLite / In-Memory	Session-based

1. Executive Summary

TaskFlow is a lightweight, full-stack task management web application built with ASP.NET Core MVC targeting .NET 8. The application was originally scaffolded with Docker and SQL Server dependencies, which were subsequently replaced with an in-memory data store and SQLite, enabling the project to run on any developer machine without requiring any external services to be installed or running.

The final deliverable is a self-contained, zero-dependency web application that starts with a single **dotnet run** command. It ships with two pre-seeded demo accounts, sample projects, and tasks, providing an immediately usable experience out of the box.

2. Project Overview

2.1 Goals

The primary goals of this project were:

- Run the application locally without Docker, SQL Server, or any database server.
- Provide a clean, modern UI with a sidebar layout, dashboards, and CRUD views.
- Eliminate BCrypt/EF migration complexity by switching to in-memory storage.
- Keep the codebase minimal and easy to understand for new developers.

2.2 Technology Stack

Framework	ASP.NET Core MVC — .NET 8
Language	C# 12
Data Layer	In-Memory store (InMemoryStore static class) — no ORM required at runtime
Auth	ASP.NET Core Session middleware — cookie-based session
Frontend	Razor Views (.cshtml) with embedded CSS — no JavaScript framework
Icons	Font Awesome 6.5 (CDN)
Fonts	Plus Jakarta Sans (Google Fonts CDN)
Build Tool	dotnet CLI
Deployment	Self-hosted Kestrel on http://localhost:5000

3. Architecture

3.1 Project Structure

The project follows a standard MVC layered architecture:

Path	Purpose
src/Controllers/Controllers.cs	HomeController, AccountController, TasksController
src/Services/Services.cs	ITaskService, IUserService + in-memory implementations
src/Models/Models.cs	User, Project, ProjectTask, ViewModels, Enums
src/Views/	Razor views for all pages (Layout, Home, Tasks, Account)
src/Views/Shared/_Layout.cshtml	Global sidebar layout, CSS design system, nav
src/Program.cs	App bootstrap — DI registration, middleware pipeline
src/appsettings.json	App configuration (connection string kept for fallback)
src/TaskManager.csproj	Project file — no EF/SQLite packages (removed)

3.2 Request Lifecycle

- Browser sends HTTP request to Kestrel on port 5000.
- Session middleware checks for a valid UserId cookie.
- Router maps the URL to the appropriate Controller action.
- Controller calls the relevant Service method (ITaskService / IUserService).
- Service reads/writes to InMemoryStore (static in-process data).
- Controller passes the model to the Razor View, which renders HTML.
- Response is sent back to the browser.

4. Data Model

4.1 Entities

Entity	Key Fields	Relations
User	Id, FullName, Email, PasswordHash, Role	Has many Projects, Tasks
Project	Id, Name, Description, Status, Deadline, OwnerId	Belongs to User; has many Tasks
ProjectTask	Id, Title, Description, Priority, Status, DueDate, ProjectId, AssignedToId	Belongs to Project and optionally a User

4.2 Enumerations

Enum	Values
TaskPriority	Low (0), Medium (1), High (2)
TaskStatus	Todo (0), InProgress (1), Done (2), Cancelled (3)
ProjectStatus	Active (0), Completed (1), OnHold (2)
UserRole	Admin, Member (string-based)

4.3 In-Memory Seed Data

On every application startup, the following data is available:

Type	Count	Details
Users	2	Administrator (Admin role), John Member (Member role)
Projects	2	Sample Project, Website Redesign
Tasks	4	Various priorities and statuses across both projects

5. Features

5.1 Authentication

Login	Email + plain-text password matched against InMemoryStore.Users
Session	ASP.NET Core ISession stores UserId, UserName, UserRole in a server-side cookie
Logout	Clears the session and redirects to Login
Register	Creates a new User in InMemoryStore with Role = Member
Guards	Each controller action calls RequireAuth() — redirects to Login if no session

5.2 Dashboard

- Four KPI stat cards: Total Tasks, Completed, In Progress, Overdue.
- Recent Tasks table (last 5, ordered by creation date).
- Active Projects panel showing project name and status badge.
- Quick-access New Task button.

5.3 Task Management (CRUD)

Operation	Route	Description
List	GET /Tasks/Index	All tasks with priority/status badges, assignee avatar, overdue highlight
Create	GET /Tasks/Create	Form with title, description, priority, status, project, assignee, due date
Save	POST /Tasks/Create	Validates and adds task to InMemoryStore
Edit	GET /Tasks/Edit/{id}	Pre-filled form for updating task fields
Update	POST /Tasks/Edit	Applies changes to the in-memory task record
Delete	POST /Tasks/Delete	Removes task, confirms via JS dialog
Details	GET /Tasks/Details	Read-only view of all task fields

6. UI Design System

All styling is implemented as embedded CSS inside `_Layout.cshtml` — no external stylesheet files or CSS frameworks are required. The design uses CSS custom properties (variables) for consistent theming.

6.1 Color Palette

Variable	Hex Value	Usage
--sidebar-bg	#0f172a	Sidebar background (navy)
--primary	#3b82f6	Buttons, active states, links (blue)
--bg	#f4f6fb	Page background (light grey)
--card-bg	#ffffff	Card and table backgrounds
--success (green)	#22c55e	Done badge, active project indicator
--warning (amber)	#f59e0b	Medium priority badge
--danger (red)	#ef4444	High priority, overdue, delete actions

6.2 Pages & Components

Component	Description
Sidebar	Fixed 250px dark navy sidebar with brand logo, nav links, user avatar chip

Stat Card	White card with colored icon square, large number, and label
Data Table	Zebra-striped table with uppercase column headers and hover highlight
Badge	Pill-shaped tag for status (blue/green/grey/red) and priority (blue/amber/red)
Auth Card	Centered white card on dark gradient — used for Login and Register
Form Controls	Bordered inputs with blue focus ring; select dropdowns and datetime pickers
Detail Rows	Key-value pairs with left label column and right value column

7. Configuration & Setup

7.1 Prerequisites

.NET SDK	8.0 or later — https://dotnet.microsoft.com/download
Docker	NOT required
Database	NOT required (in-memory store)
Node.js	NOT required

7.2 Running the Application

1. Extract	Unzip project-local.zip to any folder
2. Navigate	Open a terminal and cd into project-local\src
3. Run	Execute: dotnet run
4. Open	Visit http://localhost:5000 in your browser
5. Login	Use: admin@taskmanager.com / Admin@123

7.3 Demo Accounts

Role	Email	Password	Permissions
Admin	admin@taskmanager.com	Admin@123	Full access to all tasks and projects
Member	john@taskmanager.com	Member@123	Can view and manage assigned tasks

8. Changes Made to Original Project

File / Area	Original	Changed To
docker-compose.yml	SQL Server service definition	Dummy comment file — not needed
Dockerfile	Multi-stage .NET build	Dummy comment file — not needed
Program.cs	EF Core + SQLite + DB migrations	No DB — service registration only
TaskManager.csproj	EF Core + SQLite NuGet packages	Removed all DB packages
Data/AppDbContext.cs	EF Core DbContext class	Deleted
Migrations/	EF migration files	Deleted
Services/Services.cs	BCrypt auth + EF DB queries	In-memory store + plain-text auth
Views/Shared/_Layout	Basic/missing layout	Full sidebar + CSS design system
Views/Account/	Basic forms	Styled auth cards with demo hints
Views/Home/	Raw JSON / missing	Dashboard with stat cards
Views/Tasks/	Basic scaffolded views	Polished tables, forms, detail page
Controllers.cs	API controller (raw JSON)	Removed — MVC only

9. Known Limitations

- Data is not persisted between restarts — all changes are lost when dotnet run is stopped.
- Passwords are stored and compared as plain text — not suitable for production.
- No role-based access control enforcement in views — both roles see the same UI.
- No pagination on the Tasks list — all tasks are loaded at once.
- Static files warning (wwwroot not found) is cosmetic only — the app runs correctly.
- Project selection on Create/Edit Task uses a raw integer ID field, not a dropdown.

10. Recommended Future Improvements

Priority	Improvement	Notes
High	Persistent storage	Add SQLite or PostgreSQL with EF Core migrations
High	Password hashing	Replace plain-text with BCrypt or ASP.NET Identity
Medium	Project dropdown on forms	Replace ProjectId text input with a select list

Medium	Role-based UI	Hide admin-only actions from Member role in views
Medium	Task filtering & search	Add filter by status, priority, assignee on list page
Low	Pagination	Add server-side pagination for large task lists
Low	Email notifications	Notify assignees when tasks are created or updated
Low	File attachments	Allow users to attach files to tasks